ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/344254901

A Compiler Comparison in the RISC-V Ecosystem

Conference Paper · September 2020

DOI: 10.1109/COINS49042.2020.9191411

CITATIONS
18

READS
4,337

3 authors:

Mehrdad Poorhosseini
Carl von Ossietzky University of Oldenburg
5 PUBLICATIONS **94** CITATIONS

SEE PROFILE

Wolfgang Nebel
Institute for Information Technology
266 PUBLICATIONS **1,958** CITATIONS

SEE PROFILE

Kim Grüttner
German Aerospace Center (DLR)
136 PUBLICATIONS **756** CITATIONS

SEE PROFILE

All content following this page was uploaded by Mehrdad Poorhosseini on 16 September 2020.

The user has requested enhancement of the downloaded file.

A Compiler Comparison in the RISC-V Ecosystem

Mehrdad Poorhosseini, Wolfgang Nebel
University of Oldenburg
Oldenburg, Germany
{mehrdad.poorhosseini, wolfgang.nebel}@uol.de

Kim Grüttner
OFFIS e.V.
Oldenburg, Germany
kim.gruettner@offis.de

***Abstract*—The GNU Compiler Collection (GCC) is the traditional compiler for most embedded systems, since it supports many different instruction set architectures (ISA) in its back-end. GCC has also been the first compiler that supported the RISC-V ISA. Since a while Clang/LLVM has gained more and more interest in the embedded software community. Recently, RISC-V is also supported in the LLVM back-end and maintained in the official LLVM release. In this paper we propose a benchmark environment for the comparison of compilers in the RISC-V ecosystem. We perform a comparison of GCC against LLVM for an embedded software benchmark considering compile time, size of the resulting binary, number of instructions and execution time. The results show that LLVM compiles faster in 88% of the experiments, while GCC and LLVM produce nearly the same binary size in 51% of the experiments. In 37% GCC wins and in 12% LLVM wins. In 94% of the experiments the difference between the resulting binary size in GCC and LLVM is +/- 5%. The execution time analysis shows that in 42% of the experiments GCC and LLVM have nearly the same execution time clock cycles while in 40% GCC wins and in 18% LLVM wins.**

***Index Terms*—compiler, embedded software, RISC-V, benchmarking, timing measurement**

I. Introduction

For some years now, commercial processors have dominated the market for embedded systems. ARM, Atmel-AVR and other RISC-based commercial processor families are particularly noteworthy. These processors are also found in many IoT devices, other resource and energy constrained embedded systems. With the availability of a number of different RISC-V instruction set implementations [1] for the first time a processor architecture comparable in terms of commercial processors for embedded systems became freely available. Here in particular the PULP platform project [2] is to be called, which sat down to the goal to offer differently complex RISC-V implementations. These consist of differently complex data paths and internal parallel processing units (e.g. the processor pipeline). Furthermore, these different RISC-V processor variants can be integrated in different combinations to multi-processor or multi-core configurations. For all processor variants synthesizable models exist, which can be used for a realization as ASIC (SoC) with low-power techniques. The avaibale scalebility and flexibitity w.r.t. custom instructrion extension makes RISC-V cores highly attaractive for the usage in future resource constrained IoT and embedded systems. Furthermore, different commercial companies like SiFive, Western Digital, MicroChip and T-Head (Alibaba group) are already offering RISC-V core implementations[1].

Regarding the available software toolchain for RISC-V[2], compiler support for C and C++ is provided through GCC and Clang/LLVM. Furthermore, Linux and a list of embedded real-time operating system can be built for RISC-V platforms.

This work is supported by the German Research Foundation through the Research Training Group (DFG GRK 1765) SCARE (http://www.scare.uni-oldenburg.de).

[1]See for a list of available RISC-V core implementations https://riscv.org/risc-v-cores/.

[2]A full overview of the RISC-V software ecosystem is available at https://github.com/riscv/riscv-software-list.

For a long time the GNU Compiler Collection (GCC) has been the de-facto standard compiler for embedded systems. The main reason was its support for many embedded processors and microcontrollers and its place as the official compiler for GNU/Linux. The main reason for this success was the open-source model and the support from the GNU/Linux community. Today GCC is one of the most mature compilers for all kinds of computer systems from the embedded to the HPC spectrum. GCC has a long history that goes back to 1987. Also for RISC-V GCC was the first available compiler and is still the ”default” one.

Clang/LLVM [3] is the new ”challenger” for GCC. LLVM is an open source software developed in the University of Illinois/NCSA. The initial version was released in 2003. The name of LLVM was first abbreviated from Low Level Virtual Machine. Since LLVM is not limited to the creation of a virtual machine, the meaning behind this abbreviation has been abandoned. Clang is the C, C++, and Objective-C front-end for LLVM. Recently, back-end support for RISC-V has been added into the Clang/LLVM release [4].

With the availability of two different mature compilers for RISC-V processor, one interesting question is which compiler offers the best performance w.r.t. binary size and execution time. Since this question cannot be answered in general, because it is highly software application and hardware platform dependent, different compilers (which are also evolving in the future) have to be tested for the specific context. For this reason, the main contribution of this paper is a compiler benchmark approach for the comparison of compilers in the RISC-V ecosystem. Currently, this paper focuses on the comparison between GCC and Clang/LLVM for small bare-metal benchmark programs as a representative for typical workloads on resource-constrained IoT nodes and other embedded systems. In such systems, we have a limited amount of memory in SoCs so even small percentage memory consumption and execution time matters.

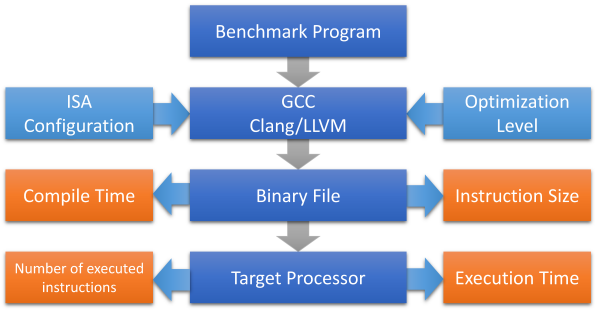


Fig. 1: Overview of the benchmark framework.

Figure 1 provides a high-level overview of the proposed compiler benchmark framework. In this framework a benchmark program is compiled for different RISC-V ISA configurations and different optimization levels. Comparison of the compilers is performed for compile-time, the resulting binary size, number of executed instruc-

tions using ISA simulator and execution time on a real target. In this work, measurements have been done on PULPissimo which is a single-core platform based on RISCY core or Ibex. In future work the resulting execution time on a 64 bit target processor will also be measured and compared.

The paper is organized as follows: In the next section related work in the area of RISC-V compiler comparison and benchmarking will be analyzed. Section III introduces our overall benchmarking approach. In the following section IV this approach gets instantiated for a selection of embedded real-time benchmarks, ISA configurations and optimization levels. Section V describes the simulations and measurements results. Finally, a conclusion about the current findings in the comparison of GCC vs. Clang/LLVM for the chosen benchmark programs will be discussed. The paper closes with a conclusion and discussion of future work.

II. Related Work

Markall [5] shared a blog post and reported a brief code size comparison between GCC and LLVM for RISC-V. In this work the `-Os` flag was used just in base ISA and the shared results were very limited whereas in our work we utilized all available standard ISA configuration flags, which indicates that our work investigated more detailed aspects of the architecture and optimizations.

Bjareholt et al. [6] evaluate the performance of GCC and Clang/LLVM in the RISC-V ecosystem by using the Freedom E310 processor on the Hifive 1 board. This comparison has been made with GCC 7.1 and LLVM 3.9. The focus was more on analyzing the performance of GCC against LLVM by using Dhrystone and coremark benchmarks. The main drawback of this work is that LLVM in this early version was still in the development process for RISC-V, while GCC already had full RISC-V support.

Ye [7] give a brief introduction to the LLVM compiler and compares the performance of LLVM with the portland Group Compiler (PGI) and GCC by using three benchmarks (GADGET-Z, LAMMPS, and HELIUM) for high-performance computing. This work used an old version of LLVM and focused more on the performance comparison, while our research used the latest version of both compilers and included also different RISC-V ISA configurations with the evaluation of different dimensions.

Hytonen [8] aimed at explaining in which cases GCC or LLVM performs better and shall be used. Furthermore, the the possibility of migration from GCC to LLVM by using different Clang and GCC versions was checked. Migration from GCC to LLVM is not in the scope of our paper and additionally, our focus is also on benchmarking for different RISC-V ISA configurations, while in Hytonen [8] no specific RISC-V ISA was mentioned.

Kim et al. [9] assessed the performance of LLVM and GCC on the ARM platform, whereas our comparison is on the RISC-V platform. Moreover, our research focused more on analysis of binary size and compile time rather than performance aspects. Anyhow, this paper could be considered for repeating these experiments for RISC-V and compare in more detail under which conditions GCC is better or LLVM performs better on ARM vs. RISC-V processors.

III. Overall Approach

We are establishing a framework where we want to compare the different compilers with respect to different degrees of freedom. This framework currently supports two dimensions (ISA configuration and optimization levels), but is extendable for supporting more dimensions. As analysis metrics we used binary size and compile time in this framework to compare two official RISC-V compilers. The binary size has been selected as one metric for the comparison, because in embedded systems memory size (especially on-chip SRAM) is a very important and expensive resource of a processing system. On the other hand, we are going to analyze the execution time in the future. And of course, when running the binary on a RISC-V target processor, we also have to deal with the available memory size. Compile time is another metric that has been chosen for comparison, because on one side it is important to see which compiler is faster and on the other side we can recognize whether the code size affects the compile time or not. For small embedded benchmark programs, as used in this work, the absolute compile time is not really an issue, but for larger systems this can become much more important. Finally, measuring the execution time on the target processor completes this comparison.

The idea of this compiler benchmark framework is finding out the advantages of both compilers with focusing on the evaluation of the young Clang/LLVM improvement against the mature and traditional GCC compiler, which has been used in RISC-V ecosystem (riscv-gnu-toolchain) so far. In this regard, we provided a script for our framework which is independent of any Instruction Set Architecture (ISA), because it could be applied to other ISAs like x86. However, in this project, we instantiate this framework for the generic RISC-V architecture. The measurement was performed on all possible (i.e. compiler supported) ISA configurations for a fair comparison with respect to compilation duration and binary size. A binary file was generated for each combination of the ISA configurations and optimization levels. This binary file consists of code and data. Code contains instructions of a benchmark source code that can change for each configuration. Data is used for initialization and will be copied to RAM, it has a unique value for almost all configurations in each benchmark. Therefore, we didn't consider data size as a parameter for compiler comparison but only consider the code size when referring to the binary size in this paper.

Figure 2 depicts the architecture of our benchmark framework as the first prototype for this paper. Currently this framework can compare two, but indeed different compilers, not only GCC against LLVM. In future work we are going to extend this toward a comparison between more than two compilers. In addition, arbitrary benchmark programs can be used with this framework. As long as these benchmark programs are self-contained programs that run on a bare-metal configuration of the processor, it will be easy to add new benchmarks to this framework for a more precise comparisons in the targets. The distinguished property of this evaluation scheme is its extendability.

IV. Benchmarking

A. Setup

We successfully instantiated our framework for the RISC-V architecture. The TACLeBench [10] benchmark collection has been selected for our experiments. These benchmarks are processor-independent and can be applied for any kind of target processor. In addition, TACLeBench is a good collection of embedded benchmarks for real-time systems. The reason that we have chosen TACLeBench is that we are focusing on the usage of LLVM and GCC for RISC-V in the embedded domain and in the future we want to focus more on the timing predictable computing. In this paper we chose 21 kernel benchmarks but in our framework we are able to do this benchmarking approach for all benchmarks in TACLeBench suite available in github. So in the future we can extend this easily to larger benchmark suites such as SPEC CPU2017 [11] and AnghaBench [12].

Our benchmarking approach is fully automated to easily reproduce the comparison for new compilers, new target processors, new

ISA configurations and other compiler options. Currently we are supporting all possible RISC-V ISA configurations. In the following a selection of these configurations have been introduced but more could also be added in the future:

- `rv32i` base integer instruction set
- `rv32if` with standard extension for single-precision floating-point
- `rv32im` with standard extension for integer multiplication and division
- `rv32ic` with standard extension for compressed instructions
- `rv32imc` with standard extension for integer multiplication and division & compressed instructions
- `rv32ifc` with standard extension for single-precision floating-point & compressed instructions

As optimization levels we are supporting the following five different optimization levels:

- `Os` enables all `O2` optimizations except those that often increase code size.
- `O0` reduces compilation time and makes debugging produce the expected results (default compiler setting).
- `O1` enables code size and execution time reduction, without performing any optimizations that take a great deal of compilation time.
- `O2` enables nearly all supported optimizations that do not involve a space-speed tradeoff. As compared to `O0`, this option increases both compilation time and the performance of the generated code.
- `O3` turns on all optimizations specified by `O2` and also turns on more optimizations that have an impact on execution time.

These parameters were declared with argument variables in the automation mechanism in order to allow testing the different possible configurations by using available compiler switches. In this project we compiled benchmarks using GCC 8.2.0 which is the latest version in riscv-gnu-toolchain so far as well as LLVM 10 which is the latest LLVM that officially support RISC-V (LLVM 9 is the first version that has official RISC-V support).

B. Code size and compile time

After generating the binary file, the `size` and `time` commands are executed for measuring the code size and compile time for a specific compiler. The following commands are used as a part of the automation infrastructure to generate the binary files and extract the code size and compile time.

```
$ time riscv32-unknown-elf-gcc -O"$opt" -march="
    $march" -mabi=ilp32 -o benchname"$opt"
    $source_code
$ report+=$(size benchname"$opt")

$ time clang-10 -O -c --target=riscv32-unknown-elf
    $input_file -march="$march" -mabi=ilp32 -O"$opt"
     && riscv32-unknown-elf-gcc benchname.o -o
    benchname"$opt" -march="$march" -mabi=ilp32 -O"
    $opt"
$ report+=$(size benchname"$opt")
```

Listing 1: Commands used for generating the binary files with GCC and Clang/LLVM.

C. Number of executed target instructions

After the compilation of binaries and completing static analysis of the software properties, we can begin the dynamic analysis by executing the codes on Spike [3]. Spike is a RISC-V ISA simulator which is able to count the number of executed instructions. It supports the 32 and 64 bit ISA, as well as combinations of the common extensions ”m”, ”a”, ”f”, ”d”, ”c”. It also offers a standard RISC-V debug interface to the simulated processor, exposed via a simulated JTAG link on the Remote Bitbang Protocol (rbb) [13]. Measurements on Spike are performed by uploading the linked binary into the running simulator using the debug interface. To do this, we use OpenOCD and gdb. The following commands are used to count the number of instructions:

```
$ spike -H --rbb-port=9824 --isa=rv32imafc -
    m0x80000000:0x10000000 \
$ /path/to/unimportant.elf
$ openocd -f spike.cfg
$ ./execute /path/to/real.elf
```

Listing 2: Commands used for Spike simulation.

D. Execution time on real hardware

Measuring the execution time of each benchmark on a RISC-V target is the most important part of our dynamic analysis. In order to execute the benchmarks, we use PULPissimo [14] which is a platform as part of the PULP project. RISCY is an in-order, single-issue core with 4 pipeline stages that has full support for RV32IMFC. We used an emulation of the PULPissimo platform on the Digilent Genesys-2 FPGA development board. The default configuration is to use RI5CY [15] core and it runs at 20 MHz. Measurements on PULPissimo are performed by uploading the binaries into the platform using the debug interface. Afterward, the execution time is measured by counting the number of clock cycles between the beginning to the end of each benchmark run. This measurement is repeated in a loop to see the possible execution time variations. For this simple processor there were no noticeable variations. The following commands are used for measuring the execution time on the RISCY.

```
$ ./init # program the FPGA
$ openocd -f pulpissimo/fpga/pulpissimo-genesys2/
    openocd-genesys2.cfg
$ ./execute /path/to/real.elf
```

Listing 3: Commands used for measuring the execution time.

E. Automation

Finally, the output of our benchmark suite is a report file and the raw results are stored in a CSV format for further analysis and comparison between the compilers. For interested readers that want to know more about our automation infrastructure or the raw data of this analysis, and for those who want to reproduce the experiments, everything will be publicly available at github, if the paper should be accepted. [4].

Figure 2 depicts how our analysis flow for the compiler comparison has been implemented. Furthermore, it demonstrates how tables and plots have been generated and how we can analyze this data. In the first phase the tree branches between the benchmarks A to U (see Table I for our selection of the TACLeBench kernel benchmarks). For each benchmark the tree further branches into the different ISA configuration and then into the different compiler optimizations, described above. For each of these paths the GCC and the LLVM compiler will be used.

In the following analysis phase we combine the results per benchmark and ISA configuration. This combined analysis is conducted for the binary (code) size, compile time, number of instructions (Spike)

[3]https://github.com/riscv/riscv-isa-sim.

[4]https://github.com/Mehrdad6991/compiler_comparison

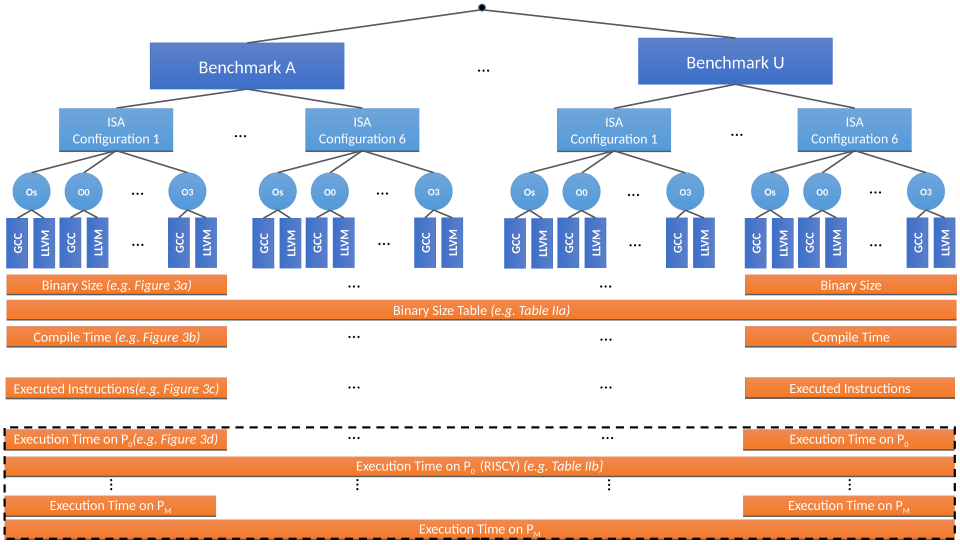


Fig. 2: Explanation of how the analysis has been done for the compiler comparison in a flow.

and execution time. An example of this first comparative analysis can be found in Figure 3a, Figure 3b , Figure3c and Figure3d (in all figures smaller numbers means better).

V. Results

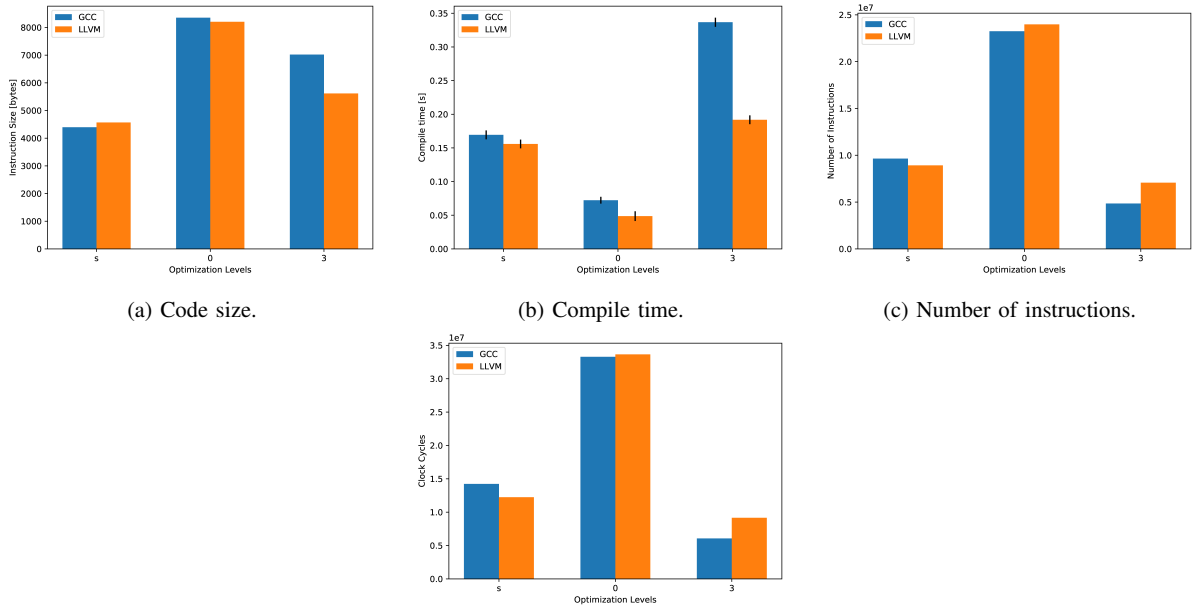
Figure 3a shows an example of an usual trend for many cases of the code size comparison of these experiments. It belongs to the `md5` benchmark, which is used for multiply-add with complex vectors. It could be understood from Figure 3a that, the number of instructions for GCC in `Os` and `O0` is approximately equal to LLVM. For `O3` this number is smaller for LLVM, which indicates GCC performs better in lower level optimizations, which is more interesting for real-time workloads. Furthermore, LLVM performs better in `O3`, which is used more for high-performance workloads. Additionally, what we can infer from this bar chart is that, although LLVM is new in the RISC-V ecosystem, it performs very close to the GCC and it also performs better than GCC in `O3`. Figure 3b compares the compilers in terms of compilation duration. In most of the cases in these experiments, LLVM compiled benchmarks faster, especially for larger benchmarks. It can be seen in the bar charts that LLVM compile time is noticeably shorter in all different optimization levels. It is also significant that LLVM is much faster than GCC in `O3`. Figure 3c represents the number of executed instructions for both compilers. It could be understood from this figure that both compilers produce approximately the same number of instructions in `Os` and `O0` but GCC produce less in `O3`. Therefore, analyzing this result indicates that the execution time for these configurations should also follow this trend. In Figure 3d obviously this assumption which was based on ISA simulation is proven and the execution time behavior is the same as the Spike result.

In this work, all different valid configurations and options have been investigated and many plots were generated but because it's not possible to show all plots, we provide the above mentioned figures as examples. In order to conclude the comparison we are introducing a second analysis level, which allows to express the results of all previous mentioned bar plot in a single table. This can be seen in Table IIa and Table IIb, which provide a compact overview over all different configurations and benchmark programs that are used in this research. In order to make this huge amount of data readable, we have used the following encoding scheme. A difference between compilers in percentage (basline is GCC) is used to show the behavior of each compiler with more details. In addition, color code is used as visual support to distinguish which compiler performs better at a glance. Black is assigned to all configurations where both compilers behave almost the same with respect to the 5% difference in either direction. Red is assigned to configurations where LLVM beats (i.e. smaller binary or faster execution time) GCC. Finally, blue is assigned to configurations where GCC beats LLVM. For instance, it can be obviously seen that GCC is the "winning" compiler in execution time comparison. particularly for jfdctint (benchmark L) GCC in most cases executed more than 200% faster.

Regarding the overall analysis of binary (code) size comparison between both compiler in Table IIa we can see that in 51% of the cases there is not much difference between GCC and LLVM. In 37% of the cases GCC wins and in 12% LLVM wins. When only considering `O0` optimization levels from the experiments, we arrive at an interesting conclusion that indicates that in 94% of these experiments both compilers have the same code size with respect to a 5% tolerance.

In order to measure the compile time it is important to repeat the measurement with a reasonable number of iterations. We repeat each configuration 100 times on a Intel® Core™ i5-8250U CPU @ 1.60GHz $\times$ 8 for all benchmarks to increase the precision and get a more accurate estimation. However, it is easy to change this number to any larger number for another comparison because, as mentioned in the previous section, this framework is extendable not only with adding new comparison dimensions but also for different comparisons in other architectures. Altogether, the results demonstrate that in 74% of the cases the compile time of LLVM is shorter. Particularly for larger benchmarks like `st` and higher optimization levels like `O3`. For instance, when considering only `O3` optimization levels in the experiments, LLVM is faster than GCC in 88% of the experiments, which indicates the better performance of LLVM in the `O3` optimization level.

The ISA simulation results generated by Spike indicate that in 46% of the experiments both compilers produce roughly equal instructions. In 14% of experiments LLVM produce less number of instructions and in 40% GCC performs better with fewer instructions. We could not display the compile time and Spike tables due to lack of space in this paper but for interested readers, all results will be publicly



(a) Code size.

(b) Compile time.

(c) Number of instructions.

(d) Execution time on RISCY.

Fig. 3: One example of the code size, compile time, number of instructions and execution time comparison between LLVM and GCC for the md5 benchmark, the RV32IMC ISA configuration and optimization all levels.

available on Github.

Regarding the execution time comparison, Table IIb demonstrates that in 42% of the experiments binary files of the benchmarks executed in nearly the same clock cycles while GCC wins in 40% of the experiments and LLVM beats GCC in 18%. In most cases, the results are optimization independent which means if a compiler beats the other one in a specific ISA configuration it wins in all optimization levels.

Although in most cases our timing measurements derive that GCC performs better than LLVM, in many cases both compilers have the same execution times and in some of them LLVM executed faster. Thus, LLVM as a young compiler in the RISC-V ecosystem has positive results. However, what we can infer from the results of these two tables is that, when LLVM performs better in most cases the number is less than 20% but whenever GCC is the winning compiler, the difference is significantly abundant. Sometimes GCC performs more than 200% better than LLVM which indicates superb power of GCC comparing LLVM in execution time analysis. In addition, by statistically analyzing Spike and execution time results we can identify that 85% of Spike simulation results are the same as execution times on real target. This confirms that Spike results could be reliably considered.

VI. Conclusion & Future Work

In this paper we have presented our compiler benchmarking approach and applied in on the comparison between GCC and LLVM in the RISC-V ecosystem. The comparison showed that LLVM and GCC are very close to each other regarding the resulting binary sizes. In 94% of the experiments, the difference between resulting binary sizes in GCC and LLVM is in the area of only 5% difference. Furthermore, it is worth mentioning that GCC, as expected before the start of this work, is not always better regarding output binary size optimization. In 12% LLVM wins over GCC. This observation leads us to the conclusion that designers for future RISC-V based embedded systems with tight memory constraints should investigate the results of both compilers. The timing measurement was firstly approximated by counting the number of executed instructions for each benchmark with different configurations using Spike ISA simulator. Secondly, timing measurement was performed by executing each benchmark with different configurations on the RISCY processor. In 42% both compilers executed in nearly the same number clock cycles, whereas

ID	Name	Description	Code Size [LOC]	Origin
A	binarysearch	Binary search of 15 integers	47	SNU-RT
B	bitonic	Bitonic sorting network	52	MiBench
C	bsort	Bubblesort program	32	MRTC
D	complex_updates	Multiply-add with complex vectors	18	DSPStone
E	countnegative	Counts signes in a matrix	35	MRTC
F	deg2rad	Convert from degrees to radians	38	MRTC
G	fac	Factorial function	21	MRTC
H	filterbank	Filter bank for multirate signals	75	StreamIt
I	fir2dim	2-dimensional FIR filter convolution	75	DSPStone
J	iir	Biquad IIR 4 sections filter	27	DSPStone
K	insertsort	Insertion sort	35	SNU-RT
L	jfdctint	Discrete-cosine transformation on a 8x8 pixel block	123	SNU-RT
M	lms	LMS adaptive signal enhancement	51	SNU-RT
N	ludcmp	LU decomposition	68	SNU-RT
O	matrix1	Generic matrix multiplication	28	DSPStone
P	md5	Message digest algorithm	344	NetBench
Q	minver	Floating point matrix inversion	141	SNU-RT
R	prime	Prime number test	41	MRTC
S	rad2deg	Convert from radians to degrees	37	MiBench
T	recursion	Artificial recursive code	18	MRTC
U	st	Statistics calculations	90	MRTC

TABLE I: Selected TACLeBench kernel benchmarks.

Architecture	opt	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	Os	+11	+10	+13	+4	+26	+3	0	0	+2	+1	+7	+3	-11	+1	+16	+7	+1	+17	+4	+2	+1
	O0	+3	+1	+2	-1	+7	0	0	+9	+2	+1	+7	-15	-2	0	+7	-2	+13	+1	0	0	+1
RV32I	O3	-31	-39	+10	-17	+51	+3	0	+18	+58	+10	+12	+5	-13	-32	+7	-15	-25	+2	+4	-6	-10
	Os	+11	+10	+13	+7	+26	+5	0	+8	+8	+4	+7	+3	-15	+1	+16	+7	+1	+17	+5	+2	+9
	O0	+3	+1	+2	+1	+7	+3	0	+8	+12	+8	+7	-15	-3	0	+7	-2	+13	+1	+2	0	+2
RV32IF	O3	-31	-39	+10	-17	+51	+2	0	+39	+147	+47	+12	+3	-11	-32	+7	-15	-25	+2	+2	-6	+8
	Os	+11	+9	+13	+4	+21	+3	0	0	+2	+1	+7	+5	-11	+1	+18	+7	0	+20	+4	+2	+1
	O0	+3	+1	+2	-1	+4	0	0	+8	+2	+1	+7	-5	-1	+1	+8	-2	+6	+3	0	0	+2
RV32IM	O3	-32	-39	+10	-17	+75	+3	0	+18	+58	+10	+12	+14	-12	-31	+6	-15	-25	+8	+4	-6	-8
	Os	+10	+8	+10	+3	+19	+3	0	+1	0	+1	+5	+2	-12	+1	+10	+4	+1	+13	+4	+1	0
	O0	+5	+3	+4	+1	+11	0	0	+5	+5	+3	+9	-9	0	+3	+10	-2	+9	+3	0	+2	+3
RV32IC	O3	-28	-34	+10	-14	+42	+3	0	+14	+59	+9	+10	+1	-13	-25	+4	-20	-23	+2	+4	-4	-8
	Os	+11	+7	+10	+3	+19	+3	0	+1	0	+1	+5	+3	-11	+1	+11	+4	+1	+19	+4	+1	+1
	O0	+4	+3	+4	+1	+9	0	0	+4	+5	+3	+9	-3	+1	+2	+11	-2	+5	+5	0	+2	+3
RV32IMC	O3	-28	-34	+10	-14	+63	+3	0	+14	+59	+9	+10	+9	-13	-25	+3	-20	-23	+9	+4	-4	-7
	Os	+10	+8	+10	+10	+19	+6	0	+11	+4	+4	+5	+2	-16	+1	+10	+4	+1	+13	+6	+1	+8
	O0	+5	+3	+4	+7	+11	+3	0	+5	+17	+11	+9	-9	0	+3	+10	-2	+9	+3	+3	+2	+4
RV32IFC	O3	-28	-34	+10	-7	+42	+2	0	+31	+146	+51	+10	0	-12	-25	+4	-21	-22	+2	+2	-4	+8

(a) Binary size.

Architecture	opt	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	Os	+41	-17	+4	+1	+43	+2	-26	+3	+2	+7	-17	+20	0	+7	+18	-14	+4	+1	+3	-2	+3
	O0	+43	-1	-15	0	+141	-1	+9	+1	+2	+3	-14	+226	0	+13	+39	-1	+26	+4	-1	+7	+3
RV32I	O3	+59	+29	+7	+1	+46	+2	0	+1	+1	+1	-8	+345	+2	0	+19	+46	+4	+4	+3	+19	+3
	Os	+41	-17	+4	+9	+43	-14	-26	-5	+37	+31	-17	+20	-1	+7	+18	-14	+4	+1	-14	-2	+34
	O0	+43	-1	-15	+5	+141	+20	+9	-22	+23	+10	-14	+226	+1	+13	+39	-1	+26	+4	+20	+7	+34
RV32IF	O3	+59	+29	+7	+40	+46	0	0	+33	+25	+6	-8	+341	0	0	+19	+45	+4	+4	0	+19	+49
	Os	-43	-39	+4	+1	-27	+2	-31	+3	+2	+7	-17	-8	0	+5	+23	-14	+2	-7	+3	-2	-1
	O0	-17	-1	-15	0	-5	-1	+12	+1	+1	+3	-14	-9	-1	0	+51	-1	+11	+3	-1	+7	0
RV32IM	O3	-30	+29	+7	+1	-45	+2	+15	+1	+1	+1	-8	-24	+1	-2	+56	+46	+3	-4	+3	+19	-1
	Os	+42	-20	+5	+1	+42	+2	-24	+2	+1	+2	-21	+21	0	+7	+19	-14	+4	+1	+3	-5	+3
	O0	+43	-1	-9	0	+146	0	+8	+2	+3	+4	-12	+227	0	+13	+50	+1	+26	+4	0	+13	+4
RV32IC	O3	+59	+30	+24	0	+45	+2	0	0	0	-4	-8	+347	+2	0	+11	+51	+4	+4	+3	+19	+3
	Os	-46	-42	+5	+1	-31	+2	-33	+2	+1	+2	-21	-10	0	+5	+14	-14	+2	-7	+3	-5	-1
	O0	-16	-1	-9	0	-1	0	+10	+2	+3	+4	-12	-11	0	+1	+71	+1	+11	+4	0	+13	0
RV32IMC	O3	-33	+30	+24	0	-46	+2	+22	0	0	-4	-8	-27	+1	-2	+51	+51	+3	-5	+3	+19	-1
	Os	+42	-20	+5	+18	+42	-14	-24	-8	+31	+11	-21	+21	-1	+7	+19	-14	+4	+1	-9	-5	+36
	O0	+43	-1	-9	+7	+146	+20	+8	-17	+29	+11	-12	+227	+2	+13	+50	+1	+26	+4	+22	+13	+35
RV32IFC	O3	+59	+30	+24	+52	+45	0	0	+37	+17	-7	-8	+348	+1	0	+11	+51	+4	+4	+5	+19	+52

(b) Execution time.

TABLE II: Summary of the code size and execution time comparisons. Black color means both compilers behave the same, blue mean GCC is better and red means LLVM is better. The basline is GCC so for instanse in execution time table +51 means GCC executed 51% faster.

in 39% GCC shows better results. In 85% Spike simulation showed a good relative accuracy when comparing the number of simulated instructions with the measurement of executed cycles on the hardware target.

Furthermore, our framework and all raw data of the executed compiler benchmarks will be made publicly available on github. This will allow other researches to use our benchmarking environment for their programs and further extend it to match their specific needs.

We are already working on the following extensions of our benchmarking environment: In the next step we are going to add execution time measurement for more targets into our environment. This will include a robust and portable time measurement concept on the target platforms, an automatic compilation and software binary deployment process. It will support the collection of measurement data from the target hardware and its integration into our existing comparison methodology.

References

[1] A. Waterman and K. Asanovic, "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA," University of California at Berkeley Berkeley United States, Tech. Rep., 2019. [Online]. Available: https://riscv.org/specifications/

[2] P. P. Project. (2019) PULP platform. [Online]. Available: https://pulp-platform.org/

[3] L. Project. (2019) LLVM Download Page. [Online]. Available: http://releases.llvm.org/download.html#10.0.0

[4] lowRISC Community Interest Company. (2019) lowrisc home page. [Online]. Available: https://www.lowrisc.org

[5] G. Markall, "RISC-V Compiler Performance Part 1: Code Size Comparisons," 2016. [Online]. Available: https://www.embecosm.com/2016/05/26/risc-v-compiler-performance-part-1-code-size-comparisons/

[6] J. Bjäreholt, "RISC-V Compiler Performance: A Comparison between GCC and LLVM/clang," 2017, bachelor's Thesis, Blekinge Institute of Technology, Faculty of Computing, Department of Software Engineering.

[7] Y. Guobin, "Getting to know the LLVM compiler," Ph.D. dissertation, The University of Edinburgh, 2011.

[8] S. Hytönen, "Replacing GCC compiler with Clang/LLVM," Vaasa University of Applied Sciences, Tech. Rep., 2018.

[9] J.-J. Kim, S.-Y. Lee, S.-M. Moon, and S. Kim, "Comparison of LLVM and GCC on the ARM Platform," in *2010 5th International Conference on Embedded and Multimedia Computing*. IEEE, Aug 2010, pp. 1–6.

[10] H. Falk, S. Altmeyer, P. Hellinckx, B. Lisper, W. Puffitsch, C. Rochange, M. Schoeberl, R. B. Sörensen, P. Wägemann, and S. Wegener, "TACLeBench: A Benchmark Collection to Support Worst-Case Execution Time Research," in *16th International Workshop on Worst-Case Execution Time Analysis (WCET 2016)*, ser. OpenAccess Series in Informatics (OASIcs), M. Schoeberl, Ed., vol. 55. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2016, pp. 2:1–2:10. [Online]. Available: http://drops.dagstuhl.de/opus/volltexte/2016/6895

[11] A. Faustino, B. Kind, J. W. M. aes, J. Rocha, B. G. aes, and F. M. Q. ao Pereira, "Automatic synthesis of compilable c benchmarks from open source repositories," Universidade Federal de Minas Gerais, Tech. Rep. 01-2020, 2020. [Online]. Available: http://cuda.dcc.ufmg.br/angha/home

[12] S. C. benchmark package home page. (2019) Spec home page. [Online]. Available: https://www.spec.org/cpu2017/

[13] R.-V. Foundation. (2019) Spike, a RISC-V ISA Simulator. [Online]. Available: https://github.com/riscv/riscv-isa-sim

[14] P. P. Project. (2019) Pulpissimo github. [Online]. Available: https://github.com/pulp-platform/pulpissimo

[15] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, "Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 10, pp. 2700–2713, 2017.

View publication stats